

GUÍA COMPLETA DE INTERFACES GRÁFICAS EN JAVA

JavaFX + FXML + Scene Builder + POO + MVC + persistencia en archivos

Objetivo: dominar los conocimientos necesarios para cumplir IL 2.1 a IL 2.8, desde configurar el proyecto hasta construir un componente completo con MVC y persistencia.

Supuesto tecnológico

Los indicadores hablan de escenas, marcado declarativo y herramienta de diseño visual. Eso encaja especialmente con JavaFX + FXML + Scene Builder. Los principios también sirven si tu curso usa otra herramienta.

Indicador	Qué debes dominar
IL 2.1	Proyecto JavaFX, módulos, ciclo de vida, Stage, Scene y Node
IL 2.2	FXML, Scene Builder, fx:id, @FXML y controladores
IL 2.3	Controles, captura de datos y eventos
IL 2.4	Patrón Modelo-Vista-Controlador
IL 2.5	ObservableList, TableView y TableColumn
IL 2.6	Navegación y validación
IL 2.7	Serialización, deserialización y DAO
IL 2.8	Proyecto integrador completo

1. POO aplicada a interfaces gráficas

Una interfaz gráfica no reemplaza la POO: la utiliza constantemente. Botones, ventanas, modelos, controladores y servicios son objetos que colaboran.

Conceptos imprescindibles

- Clase: definición de atributos y métodos.
- Objeto: instancia concreta de una clase.
- Encapsulamiento: proteger el estado con atributos privados y métodos públicos.
- Herencia: reutilizar y especializar comportamiento.
- Polimorfismo: trabajar con objetos de distintas clases mediante un tipo común.
- Composición: un objeto utiliza otros objetos; es muy común en MVC.
- Interfaces: contratos de comportamiento que una clase implementa.

```
public class Cliente {  
    private String nombre;  
    private String correo;  
  
    public Cliente(String nombre, String correo) {  
        this.nombre = nombre;  
        this.correo = correo;  
    }  
  
    public String getNombre() { return nombre; }  
    public String getCorreo() { return correo; }  
}
```

Regla mental: la vista muestra e interactúa; el controlador coordina; el modelo representa datos/reglas; el DAO almacena.

2. IL 2.1 — Configuración, módulos, ciclo de vida y escenas

Debes poder: configurar una aplicación JavaFX, identificar sus módulos, explicar el inicio de la aplicación y diferenciar Stage, Scene y Node.

2.1 JavaFX

JavaFX es un framework para interfaces de escritorio. Proporciona ventanas, controles, escenas, eventos, estilos y FXML.

Elemento	Significado	Ejemplo
Stage	Ventana principal del sistema operativo.	Ventana de la aplicación
Scene	Contenido mostrado dentro del Stage.	Pantalla de login
Node	Elemento visual de una Scene.	Button, Label, TextField
Parent	Nodo que contiene otros nodos.	VBox, HBox, BorderPane

2.2 Jerarquía visual

```
Stage
├── Scene
│   ├── Root (BorderPane, VBox, etc.)
│   │   ├── Label
│   │   ├── TextField
│   │   └── Button
```

2.3 Ciclo de vida básico

```
public class Main extends Application {
    @Override
    public void start(Stage stage) throws Exception {
        FXMLLoader loader = new FXMLLoader(
            getClass().getResource("principal.fxml"));
        Scene scene = new Scene(loader.load());
        stage.setTitle("Mi aplicación");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

launch() inicia JavaFX y termina llamando a start(Stage). En start normalmente cargas FXML, construyes la Scene, configuras el Stage y llamas a show().

2.4 Módulos

```
module com.ejemplo.app {
    requires javafx.controls;
    requires javafx.fxml;

    opens com.ejemplo.controller to javafx.fxml;
    exports com.ejemplo;
}
```

javafx.controls contiene controles; javafx.fxml permite trabajar con FXML. opens suele ser necesario para que FXML acceda al controlador mediante reflexión.

3. IL 2.2 — FXML, Scene Builder y controladores

FXML permite describir la interfaz de forma declarativa. Scene Builder facilita editar visualmente ese FXML.

3.1 Marcado declarativo

En lugar de programar paso a paso cómo crear cada componente, declaras qué componentes existen y qué propiedades tienen.

```
<VBox spacing="10.0"
    xmlns:fx="http://javafx.com/fxml"
    fx:controller="com.ejemplo.controller.LoginController">
    <Label text="Usuario"/>
    <TextField fx:id="txtUsuario"/>
    <Label text="Contraseña"/>
    <PasswordField fx:id="txtClave"/>
    <Button text="Ingresar" onAction="#iniciarSesion"/>
</VBox>
```

3.2 Conexión FXML ↔ controlador

```
public class LoginController {
    @FXML private TextField txtUsuario;
    @FXML private PasswordField txtClave;

    @FXML
    private void iniciarSesion() {
        String usuario = txtUsuario.getText();
        String clave = txtClave.getText();
    }
}
```

fx:id identifica un componente. @FXML marca campos/métodos que FXML conectará. fx:controller indica qué clase controla esa vista. onAction vincula un evento a un método.

3.3 Flujo con Scene Builder

1. Crear/abrir el FXML.
2. Elegir el contenedor raíz.
3. Arrastrar controles.
4. Configurar propiedades.
5. Asignar fx:id a los elementos necesarios.
6. Asignar eventos como onAction.
7. Indicar Controller Class.
8. Guardar y ejecutar.

Error típico: si FXML apunta a un controlador incorrecto, o el fx:id/nombre del método no coincide, la vista puede fallar al cargarse.

4. IL 2.3 — Componentes, datos y eventos

Debes capturar información, mostrarla, modificarla y reaccionar a las acciones del usuario.

Control	Uso
Label	Mostrar texto
TextField	Texto corto
PasswordField	Contraseña
TextArea	Texto largo
Button	Ejecutar acción
CheckBox	Activar/desactivar opción
RadioButton	Elegir una opción del grupo
ComboBox	Seleccionar de una lista
DatePicker	Elegir fecha
TableView	Mostrar colecciones
Alert	Mensajes y confirmaciones

4.1 Leer datos

```
String nombre = txtNombre.getText();
boolean activo = chkActivo.isSelected();
String ciudad = cmbCiudad.getValue();
lblResultado.setText("Hola " + nombre);
```

4.2 Eventos

```
@FXML
private void guardar() {
    String nombre = txtNombre.getText();

    if (nombre.isBlank()) {
        mostrarError("El nombre es obligatorio");
        return;
    }

    // procesar la acción
}
```

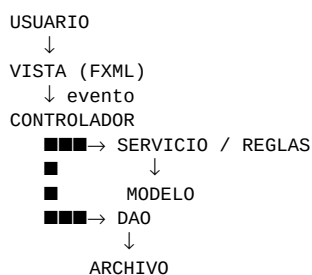
Un botón no sirve solo por existir: debe tener un evento que provoque una acción. Si la acción es compleja, delega la lógica a servicios/modelo en lugar de hacer todo en el controlador.

5. IL 2.4 — Patrón Modelo-Vista-Controlador

MVC separa responsabilidades. No basta con crear carpetas llamadas model, view y controller: cada parte debe tener una función clara.

Capa	Responsabilidad	Ejemplos
Modelo	Datos y reglas del dominio	Cliente, Reserva, Producto
Vista	Presentación e interacción	FXML, Scene Builder
Controlador	Recibir eventos y coordinar	LoginController
DAO	Acceso al almacenamiento	ClienteDAO
Servicio	Lógica de negocio reutilizable	ReservaService

5.1 Flujo MVC



5.2 Ejemplo: guardar cliente

1. El usuario llena el formulario.
2. Pulsa Guardar.
3. El controlador recibe el evento.
4. El controlador valida.
5. Se crea/modifica un Cliente.
6. El servicio aplica reglas si existen.
7. El DAO guarda.
8. La vista se actualiza.

Regla para memorizar: Vista muestra → Controlador coordina → Modelo representa → DAO almacena.

6. IL 2.5 — Colecciones y TableView

TableView presenta una colección de objetos en filas y columnas.

6.1 Modelo

```
public class Producto {
    private final String nombre;
    private final double precio;
    private final int stock;

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre; this.precio = precio; this.stock = stock;
    }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }
    public int getStock() { return stock; }
}
```

6.2 FXML

```
<TableView fx:id="tablaProductos">
    <columns>
        <TableColumn fx:id="colNombre" text="Nombre"/>
        <TableColumn fx:id="colPrecio" text="Precio"/>
        <TableColumn fx:id="colStock" text="Stock"/>
    </columns>
</TableView>
```

6.3 Controlador

```
@FXML private TableView<Producto> tablaProductos;
@FXML private TableColumn<Producto, String> colNombre;
@FXML private TableColumn<Producto, Number> colPrecio;
@FXML private TableColumn<Producto, Number> colStock;

private ObservableList<Producto> productos =
    FXCollections.observableArrayList();

@FXML
private void initialize() {
    colNombre.setCellValueFactory(d ->
        new SimpleStringProperty(d.getValue().getNombre()));
    colPrecio.setCellValueFactory(d ->
        new SimpleDoubleProperty(d.getValue().getPrecio()));
    colStock.setCellValueFactory(d ->
        new SimpleIntegerProperty(d.getValue().getStock()));
    tablaProductos.setItems(productos);
}
```

Cadena que debes entender: objetos → ObservableList → TableView → TableColumn → propiedad del objeto.

7. IL 2.6 — Navegación y validación

Una aplicación real tiene varias vistas. Debes saber cambiar de FXML y controlar los datos que viajan entre pantallas.

7.1 Cambiar de vista

```
@FXML
private void abrirRegistro(ActionEvent event) throws IOException {
    FXMLLoader loader = new FXMLLoader(
        getClass().getResource("/com/ejemplo/registro.fxml"));
    Parent root = loader.load();
    Stage stage = (Stage) ((Node) event.getSource())
        .getScene().getWindow();
    stage.setScene(new Scene(root));
    stage.show();
}
```

7.2 Pasar datos a otro controlador

```
FXMLLoader loader = new FXMLLoader(
    getClass().getResource("detalle.fxml"));
Parent root = loader.load();
DetalleController c = loader.getController();
c.setCliente(clienteSeleccionado);
```

El orden es importante: load() primero; después getController(); luego puedes enviar el objeto.

7.3 Validación

```
private boolean validarFormulario() {
    if (txtNombre.getText().isBlank()) {
        mostrarError("Nombre obligatorio");
        return false;
    }
    if (txtCorreo.getText().isBlank() ||
        !txtCorreo.getText().contains("@")) {
        mostrarError("Correo inválido");
        return false;
    }
    return true;
}

@FXML
private void guardar() {
    if (!validarFormulario()) return;
    // continuar
}
```

Dato	Regla de ejemplo
Nombre	No vacío
Edad	Entero ≥ 18
Precio	Número ≥ 0
Stock	Entero no negativo
Fecha inicio/fin	Fin no anterior al inicio

8. IL 2.7 — Persistencia, serialización, deserialización y DAO

Persistencia significa que los datos sobreviven al cierre de la aplicación.

8.1 Serialización

Serializar es convertir el estado de un objeto en una representación que puede almacenarse. Una forma clásica en Java es Serializable + ObjectOutputStream.

```
public class Cliente implements Serializable {
    private static final long serialVersionUID = 1L;
    private String nombre;
    private String correo;

    public Cliente(String nombre, String correo) {
        this.nombre = nombre; this.correo = correo;
    }
}
```

8.2 Guardar

```
public void guardar(List<Cliente> clientes) throws IOException {
    try (ObjectOutputStream salida = new ObjectOutputStream(
        new FileOutputStream("clientes.dat"))) {
        salida.writeObject(clientes);
    }
}
```

8.3 Deserializar

```
public List<Cliente> cargar() throws IOException, ClassNotFoundException {
    File archivo = new File("clientes.dat");
    if (!archivo.exists()) return new ArrayList<>();
    try (ObjectInputStream entrada = new ObjectInputStream(
        new FileInputStream(archivo))) {
        return (List<Cliente>) entrada.readObject();
    }
}
```

8.4 DAO

DAO significa Data Access Object. Encapsula el almacenamiento para que el controlador no tenga que saber cómo abrir streams ni escribir archivos.

```
public class ClienteDAO {
    public void guardar(List<Cliente> clientes) throws IOException {
        // serialización
    }
    public List<Cliente> cargar()
        throws IOException, ClassNotFoundException {
        // deserialización
        return new ArrayList<>();
    }
}
```

Ventaja: si mañana cambias archivo por una base de datos, la vista no debería necesitar conocer los detalles del nuevo almacenamiento.

9. IL 2.8 — Proyecto integrador completo

Ejemplo de requerimiento: “El usuario debe registrar productos, visualizar los existentes, validar los datos y conservarlos al cerrar y abrir nuevamente el programa.”

9.1 Estructura

```
src/main/java/com.ejemplo/  
■■■ Main.java  
■■■ model/Producto.java  
■■■ controller/PrincipalController.java  
■■■ controller/ProductoController.java  
■■■ service/ProductoService.java  
■■■ dao/ProductoDAO.java  
  
src/main/resources/com/ejemplo/  
■■■ principal.fxml  
■■■ producto.fxml
```

9.2 Flujo completo

1. Main inicia JavaFX y carga principal.fxml.
2. El controlador pide productos al DAO.
3. El DAO deserializa el archivo.
4. La colección llega a ObservableList.
5. TableView muestra los productos.
6. El usuario pulsa Nuevo.
7. Se carga producto.fxml.
8. El usuario completa y valida.
9. Se crea Producto.
10. El servicio aplica reglas.
11. El DAO persiste.
12. La vista principal se actualiza.

9.3 Modelo integrable

```
public class Producto implements Serializable {  
    private static final long serialVersionUID = 1L;  
    private String nombre;  
    private double precio;  
    private int stock;  
  
    public Producto(String nombre, double precio, int stock) {  
        this.nombre = nombre; this.precio = precio; this.stock = stock;  
    }  
    public String getNombre() { return nombre; }  
    public double getPrecio() { return precio; }  
    public int getStock() { return stock; }  
}
```

9.4 Controlador de formulario: idea

```
@FXML private TextField txtNombre;  
@FXML private TextField txtPrecio;  
@FXML private TextField txtStock;  
  
@FXML  
private void guardar() {  
    if (txtNombre.getText().isBlank()) {  
        mostrarError("Nombre obligatorio");  
        return;  
    }  
    try {  
        double precio = Double.parseDouble(txtPrecio.getText());  
        int stock = Integer.parseInt(txtStock.getText());  
        if (precio < 0 || stock < 0) {
```

```
        mostrarError("No se permiten valores negativos");
        return;
    }
    Producto p = new Producto(txtNombre.getText(), precio, stock);
    // servicio/DAO
} catch (NumberFormatException e) {
    mostrarError("Precio y stock deben ser numéricos");
}
}
```

10. Cómo demostrar cada indicador en una evaluación

IL	Evidencia que deberías poder mostrar
2.1	Proyecto ejecutable, module-info, Stage/Scene, ciclo de vida
2.2	FXML diseñado con Scene Builder y conectado a controller
2.3	Controles que capturan datos y eventos funcionales
2.4	Modelo, vista y controlador con responsabilidades separadas
2.5	TableView alimentada desde una colección
2.6	Cambio entre vistas + validación de formulario
2.7	Archivo persistente + serialización/deserialización + DAO
2.8	Todo integrado en un flujo de negocio coherente

11. Errores frecuentes

Problema	Qué revisar
FXML no carga	Ruta, fx:controller, imports y recursos
@FXML queda null	fx:id, controlador correcto y FXMLLoader
Botón no responde	onAction y nombre/firma del método
Tabla vacía	ObservableList, setItems y cellValueFactory
No cambia pantalla	Ruta del FXML y loader.load()
Datos desaparecen	DAO, ruta del archivo y guardar/cargar
NumberFormatException	El texto no es un número válido
ClassCastException	Tipo real del objeto y genéricos

12. Checklist final de estudio

- Puedo explicar Stage, Scene y Node.
- Puedo explicar el ciclo de vida básico de JavaFX.
- Sé para qué sirven `javafx.controls` y `javafx.fxml`.
- Sé qué función cumple `module-info.java`.
- Puedo crear una vista FXML con Scene Builder.
- Sé conectar `fx:id` con `@FXML`.
- Puedo programar eventos.
- Puedo capturar datos de `TextField`, `ComboBox`, `CheckBox` y `DatePicker`.
- Puedo explicar MVC sin memorizar una definición.
- Puedo llenar una `TableView` desde `ObservableList`.
- Puedo navegar entre dos vistas.
- Sé pasar un objeto a otro controlador.
- Puedo validar un formulario.
- Sé qué son serialización y deserialización.
- Puedo guardar y recuperar una colección.
- Sé por qué existe un DAO.
- Puedo explicar Vista → Controlador → Modelo/Servicio → DAO → Archivo.
- ¿Qué diferencia existe entre Stage y Scene?
- ¿Qué problema resuelve FXML?
- ¿Para qué sirve `fx:id` y qué relación tiene con `@FXML`?
- ¿Qué responsabilidad tiene el controlador?
- ¿Qué significa MVC y por qué mejora la organización?
- ¿Cómo se llena una `TableView` desde una colección?
- ¿Cómo pasas un objeto de una pantalla a otra?
- ¿Qué diferencia existe entre serialización y deserialización?
- ¿Por qué usar DAO?
- ¿Qué ocurre si `Integer.parseInt()` recibe "abc"?
- ¿Qué debería ocurrir si un formulario tiene campos vacíos?